Capítulo 1: Inicio: Sobre el Control de Versiones

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Este capítulo presenta qué es el control de versiones, por qué es útil y qué hace diferente a Git de otras herramientas. También explica cómo instalar y configurar Git por primera vez.

1.1 Acerca del control de versiones

Un sistema de control de versiones (VCS) registra los cambios hechos sobre un conjunto de archivos a lo largo del tiempo. Gracias a él es posible volver a una versión anterior, comparar cambios, saber quién modificó algo y recuperarse de errores sin perder el trabajo.

- Locales: guardan las versiones en una base de datos en el mismo computador. Son simples, pero no permiten colaborar.
- Centralizados (por ejemplo, Subversion): un único servidor almacena todas las versiones. Facilitan la colaboración, pero si el servidor falla se convierte en un punto único de falla.
- Distribuidos (por ejemplo, Git): cada usuario tiene una copia completa del repositorio con todo su historial, de modo que cualquier copia sirve como respaldo.

1.2 Breve historia de Git

Git nació en 2005 de la mano de Linus Torvalds, para gestionar el desarrollo del kernel de Linux cuando la herramienta que usaba la comunidad dejó de ser gratuita. Sus objetivos de diseño fueron: velocidad, diseño sencillo, soporte para el desarrollo no lineal (miles de ramas en paralelo), carácter distribuido y capacidad para manejar proyectos muy grandes.

1.3 Qué es Git

- Instantáneas, no diferencias: Git guarda una "foto" del estado de todos los archivos en cada confirmación.
- Casi todas las operaciones son locales: no se necesita conexión de red para consultar el historial o confirmar cambios.
- Integridad: todo se identifica mediante una suma de comprobación SHA-1, por lo que Git detecta cualquier alteración o corrupción.
- Generalmente solo añade datos, lo que hace muy difícil perder información una vez confirmada.

Los tres estados

Un archivo en Git puede estar en tres estados: modificado (cambió pero no está marcado), preparado o staged (marcado para ir en la próxima confirmación) y confirmado o committed (guardado de forma segura en la base de datos local). Esto da lugar a tres áreas: el directorio de trabajo, el área de preparación (staging area) y el directorio .git.

El flujo básico es: se modifican archivos en el directorio de trabajo, se preparan los cambios con git add y se confirman con git commit.

1.4 La línea de comandos

La línea de comandos es la única forma de ejecutar todas las funciones de Git. Las interfaces gráficas, como GitHub Desktop, implementan solo un subconjunto de ellas. Quien sabe usar la línea de comandos puede usar cualquier interfaz gráfica, pero lo contrario no siempre es cierto.

1.5 Instalación de Git

En Windows se descarga el instalador desde git-scm.com (incluye Git Bash). En macOS se puede instalar con las herramientas de línea de comandos de Xcode o con Homebrew. En Linux se usa el gestor de paquetes de la distribución, por ejemplo:

```
sudo apt install git
git --version
```

1.6 Configuración inicial

Git incluye la herramienta git config para definir variables de configuración. Existen tres niveles: --system (todo el equipo), --global (el usuario actual) y --local (un repositorio específico). Lo primero que se debe hacer es indicar la identidad, ya que se incluye en cada confirmación:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global core.editor "code --wait"
git config --list
```

El comando git config user.name permite consultar un valor específico.

1.7 Obtener ayuda

```
git help <comando>
git <comando> --help
git <comando> -h
```

Estos comandos muestran el manual completo o un resumen breve de las opciones de cualquier comando de Git.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/es/v2